

PrepME AI — Day 4 Notes (JWT Authentication & Protected Routes)

1. Current Application Architecture

By the end of Day 4, our application flow became:

```
graph TD; A[Frontend (React)] --> B[Login Request]; B --> C[Backend Route]; C --> D[Controller]; D --> E[JWT Generation]; E --> F[Frontend Stores Token]; F --> G[Protected API Request]; G --> H[JWT Verification]; H --> I[Protected Data Returned];
```

This is the foundation of authentication used in modern web applications.

2. Problem Before JWT

Before JWT:

```
graph TD; A[User Login] --> B[Backend verifies password]; B --> C[Returns success message];
```

Problem:

How does backend know
who is making future requests?

Example:

```
/profile  
/dashboard  
/settings
```

Backend needs a way to identify the user.

3. What Is JWT?

JWT stands for:

JSON Web Token

JWT is a digitally signed token that represents an authenticated user.

Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6I...
```

JWT acts like:

```
Movie Ticket  
ID Card  
Access Pass
```

After login, user receives a JWT and uses it for future requests.

4. JWT Authentication Flow

```
Login  
↓  
Verify Password
```

```
↓  
Generate JWT  
↓  
Send JWT To Frontend  
↓  
Store JWT  
↓  
Future Requests Send JWT  
↓  
Backend Verifies JWT  
↓  
Access Granted
```

This is called:

Token Based Authentication

5. Installing JWT

Package:

```
npm install jsonwebtoken
```

Import:

```
const jwt = require("jsonwebtoken")
```

Purpose:

```
Generate tokens  
Verify tokens
```

6. JWT Secret

Inside:

```
JWT_SECRET=mysecretkey123
```

Purpose:

Secret key used to sign tokens
and verify tokens

Important:

Only backend knows this secret.

Never expose JWT_SECRET to frontend.

7. Generating JWT

Code:

```
const token = jwt.sign(  
  {  
    userId: user._id,  
    email: user.email  
  },  
  process.env.JWT_SECRET,  
  {  
    expiresIn: "1h"  
  }  
)
```

Purpose:

Create signed authentication token

8. JWT Payload

Payload:

```
{
  userId: user._id,
  email: user.email
}
```

Purpose:

Store user information
inside token

Examples:

- userId
- email
- role

Never store:

Passwords
Sensitive secrets

inside JWT.

9. expiresIn

Example:

```
expiresIn: "1h"
```

Meaning:

Token valid for 1 hour

After expiration:

User must login again

10. Login Response

Backend returns:

```
{  
  "message": "Login successful",  
  "token": "eyJhbGc..."  
}
```

Important debugging lesson:

We generated token successfully but forgot:

```
res.json({  
  message: "Login successful",  
  token  
})
```

Token must be returned in response.

11. Understanding JWT Verification

Code:

```
jwt.verify(  
  token,  
  process.env.JWT_SECRET  
)
```

Purpose:

```
Check if token was created  
using the same secret key
```

Backend is NOT checking:

```
Does token contain secret?
```

Backend is checking:

```
Was token signed  
using my secret?
```

12. JWT Secret Change Scenario

Example:

Today:

```
JWT_SECRET=mysecretkey123
```

Tomorrow:

```
JWT_SECRET=newsecret999
```

Result:

```
All existing tokens become invalid.
```

Reason:

```
Tokens were signed using old secret.
```

13. Middleware Concept

New backend concept:

```
Middleware
```

Definition:

Code that runs before route logic.

Flow:

```
Request
  ↓
Middleware
  ↓
Route
```

Purpose:

Reuse logic across multiple routes.

14. Why Middleware?

Without middleware:

```
Verify token
Verify token
Verify token
Verify token
```

inside every route.

With middleware:

```
Write once
Reuse everywhere
```

15. Auth Middleware

File:

```
middleware/authMiddleware.js
```

Purpose:

```
Verify JWT before route access
```

Main responsibilities:

1. Read token
2. Verify token
3. Attach user data
4. Continue request

16. Authorization Header

Client sends:

```
Authorization:  
TOKEN
```

Backend reads:

```
req.header("Authorization")
```

Purpose:

```
Receive JWT from client
```

17. next() Function

Code:

```
next()
```

Meaning:

```
Continue to next middleware  
or route
```

Without next():

```
Request stops
```

Very important Express concept.

18. Protected Route

Created:

```
GET /api/users/profile
```

Purpose:

```
Only authenticated users  
can access this route
```

Route flow:

```
Request  
↓  
authMiddleware  
↓  
getProfile
```

19. Attaching User To Request

Code:

```
req.user = decoded
```

Purpose:

Pass authenticated user
to route controller

Later route can access:

```
req.user.email  
req.user.userId
```

20. Profile Controller

Example:

```
res.json({  
  message: "Protected route accessed",  
  user: req.user  
})
```

Purpose:

Return authenticated user data

21. Postman Testing

Learned proper testing workflow.

Wrong

Token sent as:

Query Parameter

Example:

```
/profile?Authorization=TOKEN
```

Correct

Token sent as:

HTTP Header

Example:

Authorization: TOKEN

22. Frontend Token Storage

New browser concept:

localStorage

Purpose:

Store data inside browser

Examples:

- JWT token
 - theme preference
 - settings
-

23. Save Token

Code:

```
localStorage.setItem(  
  "token",  
  response.data.token  
)
```

Purpose:

Store JWT after login

24. Read Token

Code:

```
localStorage.getItem("token")
```

Purpose:

Retrieve JWT from browser

25. Remove Token

Code:

```
localStorage.removeItem("token")
```

Purpose:

Logout user

(Implemented later)

26. Frontend Authentication Flow

Current flow:

```
Login
↓
Receive Token
↓
Store Token
```

↓
Read Token
↓
Send Token In Header
↓
Protected API
↓
Receive Profile Data

27. React Profile State

Created:

```
const [profile, setProfile] =  
  useState(null)
```

Purpose:

Store profile response
inside UI state

28. Get Profile Function

Purpose:

Read token
Call protected route
Display profile

Flow:

```
localStorage  
↓  
Authorization Header  
↓  
Backend Verification  
↓
```

Profile Response
↓
React State

29. React Conditional Rendering

Used:

```
{profile && (  
  ...  
)}
```

Meaning:

Render UI only if
profile exists

Equivalent to:

```
if(profile){  
  show UI  
}
```

30. React Form Bug

Problem:

Buttons inside:

```
<form>
```

behave like:

Submit Buttons

by default.

Result:

Page refresh
State reset

Fix:

```
<button  
  type="button"  
>
```

Important React/UI lesson.

31. JWT Storage Clarification

Current token location:

Returned by backend
Stored in localStorage

Then:

Read from localStorage
Sent in Authorization Header

Token is NOT stored in database.

32. Why JWT Works Without Storage

JWT is:

Self-contained

Backend does not remember tokens.

Instead:

```
jwt.sign()  
creates signature  
  
jwt.verify()  
checks signature
```

using the same secret.

This is called:

Stateless Authentication

33. Most Important Learning Today

Authentication is NOT:

Check email and password

Authentication is:

```
Verify user  
Generate token  
Store token  
Protect routes  
Verify token  
Control access
```

34. Current Features Completed

✅ MongoDB Connection

✅ User Schema

✅ User Creation

- ✓ Duplicate Email Validation
 - ✓ Password Hashing (bcrypt)
 - ✓ Login Verification
 - ✓ JWT Generation
 - ✓ JWT Verification
 - ✓ Middleware
 - ✓ Protected Routes
 - ✓ localStorage
 - ✓ Frontend Authentication Flow
-

35. Next Learning Goals

Next steps:

```
Logout  
Bearer Token Format  
Automatic Login Persistence  
Role Based Authorization  
Refresh Tokens  
Frontend Route Protection
```

These are advanced authentication topics built on today's foundation.